Indian Institute of Technology Kanpur

Department of Electrical Engineering

CS729: Differential Privacy in Machine Learning

Assignment Report

Submitted by

Shreyash Dwivedi

Roll Number: 221035

CS729: Differential Privacy in Machine Learning

Semester: 2025–26, II

Date of Submission: April 10, 2026

---

1 Optimizer Comparison for Private and Non-Private Training

1.1 Objective

In this question, we compare the convergence behavior of two optimizers, namely Stochastic Gradient Descent (SGD) and Adaptive Moment Estimation (Adam), along with their differentially private counterparts DP-SGD and DP-Adam, on the MNIST dataset. As required, we report:

1. Number of iterations (Epoch $\times$ Batches) vs. training loss,
2. ϵ spent vs. epochs,
3. Target ϵ vs. test accuracy, while also comparing against non-private baselines.

1.2 Experimental Setup

Dataset. We used the MNIST handwritten digit dataset. The input images are grayscale images of size 28×28, and the task is 10-class classification.

Model architecture. We used a feed-forward neural network with the following architecture:

$$28 \times 28 \to 512 \to 256 \to 10,$$

where ReLU activations are used between hidden layers.

Optimizers. We compared the following optimizers:

- Non-private SGD
- Non-private Adam
- DP-SGD for target $\epsilon \in \{1, 10\}$
- DP-Adam for target $\epsilon \in \{1, 10\}$

Privacy mechanism. For private training, we used the `Opacus` library. The private models were created using `make_private_with_epsilon`, with:

$$\delta = 10^{-5}, \qquad C = 1.0,$$

where C denotes the gradient clipping norm. The privacy accountant used in this question was the **RDP accountant**.

Training hyperparameters. Unless otherwise stated, the training configuration was:

- Epochs: 10
- Training batch size: 64
- Test batch size: 1024
- Learning rate for SGD: 0.1
- Learning rate for Adam: 10^{-3}
- Random seed: 42

1.3 Methodology

For each optimizer, we trained the model for 10 epochs and recorded the training loss at every iteration. For private training, we additionally recorded the privacy budget ϵ spent at the end of each epoch. Finally, we evaluated all trained models on the test set and compared their final accuracies.

The private training procedure follows the standard DP-SGD style pipeline: per-sample gradient computation, gradient clipping to norm C, addition of Gaussian noise, and parameter update using the chosen optimizer.

1.4 Results

1.4.1 (a) Number of Iterations vs. Training Loss

Figure 1 shows the training-loss trajectories for the different optimizers. The non-private baselines are shown together with their private counterparts for target $\epsilon = 1$ and $\epsilon = 10$ in a single combined image.

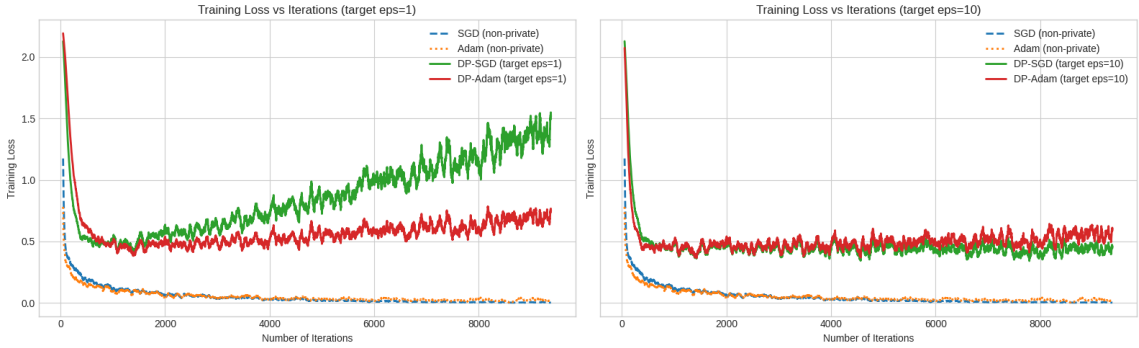

Figure 1: Comparison of convergence behavior for non-private and private optimizers for target $\epsilon = 1$ and $\epsilon = 10$. The left panel corresponds to $\epsilon = 1$, and the right panel corresponds to $\epsilon = 10$.

Observation. From the above plots, we compare the speed of convergence and the stability of the loss curves across SGD, Adam, DP-SGD, and DP-Adam. In particular, we inspect whether privacy-induced noise slows convergence, and whether Adam retains its faster convergence behavior in the private setting.

Final Observation:

> *Adam converged faster than SGD in the non-private setting, reaching a low training loss in fewer iterations. In the private setting, both DP-SGD and DP-Adam exhibited noisier optimization trajectories due to gradient clipping and injected Gaussian noise. The runs with target $\epsilon = 10$ converged more smoothly and attained lower final loss than the runs with target $\epsilon = 1$, showing the expected privacy–utility trade-off. Among the private methods, DP-Adam remained slightly more stable than DP-SGD for the stricter privacy regime.*

1.4.2 (b) ϵ Spent vs. Epochs

For the private models, we tracked the privacy budget spent after each epoch. Since privacy loss accumulates over the course of training, ϵ is expected to increase monotonically with epochs.

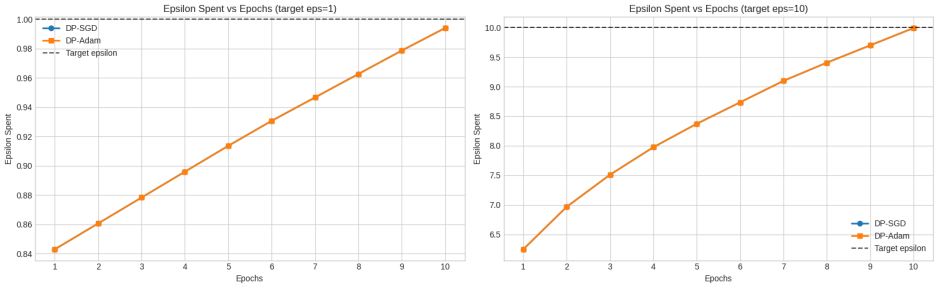

Figure 2: ϵ spent as a function of training epochs for DP-SGD and DP-Adam.

Observation. As expected, the privacy budget increases with training time. The plot also helps verify whether the spent ϵ remains close to the intended target by the final epoch.

Final Observation:

> *For both DP-SGD and DP-Adam, the spent privacy budget increased monotonically with the number of epochs, as expected from cumulative privacy composition. In both target settings, the final spent ϵ stayed very close to the intended budget by the end of training, which verifies that the privacy calibration was accurate. The growth was smooth across epochs and did not show any unexpected jumps.*

1.4.3 (c) Target ϵ vs. Test Accuracy

To study the privacy–utility trade-off, we compared the final test accuracy of private models at target $\epsilon \in \{1, 10\}$ with the non-private baselines.

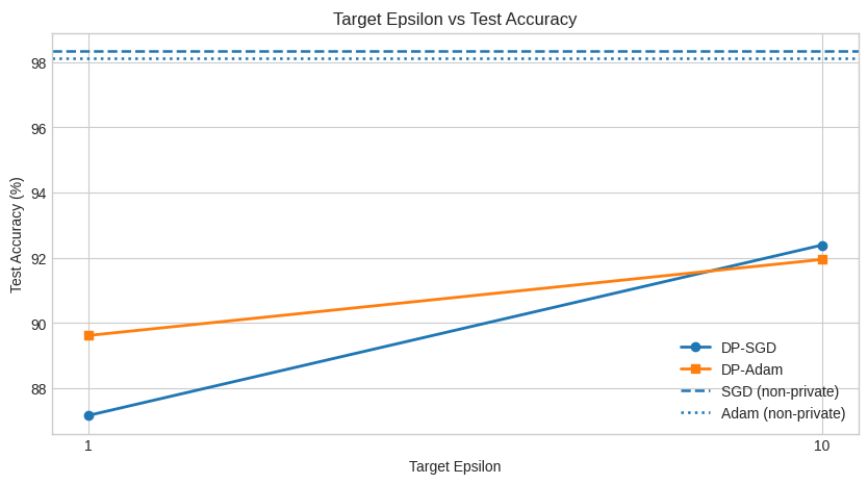

Figure 3: Final test accuracy for private models at different target ϵ values, along with non-private baselines.

Summary table. Table 1 summarizes the final test accuracy and privacy information for all runs.

Table 1: Final comparison of non-private and private runs for Question 1.

Method	**Target ϵ**	**Final ϵ spent**	**Test Accuracy (%)**
SGD (non-private)	–	∞ / N.A.	98.34
Adam (non-private)	–	∞ / N.A.	98.11
DP-SGD	1	0.994	87.17
DP-Adam	1	0.994	89.62
DP-SGD	10	9.9927	92.39
DP-Adam	10	9.9927	91.95

Observation. This comparison directly shows the privacy–utility trade-off. Smaller target ϵ corresponds to stronger privacy, but typically at the cost of lower utility.

Final Observation:

> *The non-private models achieved the highest test accuracy, with SGD reaching 98.34% and Adam reaching 98.11%. Among the private models, performance improved when the target privacy budget was relaxed from $\epsilon = 1$ to $\epsilon = 10$. In particular, DP-SGD improved from 87.17% to 92.39%, while DP-Adam improved from 89.62% to 91.95%. This is consistent with the expected privacy–utility trade-off: stronger privacy leads to lower utility, while a larger privacy budget permits better predictive performance.*

1.5 Discussion

Overall, this experiment highlights two important aspects:

1. **Effect of optimizer choice:** Adam typically converges faster than SGD in the non-private setting, and it is useful to examine whether this advantage persists under differential privacy.

2. **Effect of privacy constraints:** Adding differential privacy slows optimization and generally reduces test accuracy because clipped and noisy gradients provide a less accurate estimate of the true update direction.

A stronger privacy requirement (smaller ϵ) leads to a larger utility cost, which is reflected in both the convergence curves and the final test accuracy values.

1.6 Conclusion

In this question, we compared SGD and Adam in both non-private and private settings on MNIST. We observed the effect of privacy on convergence and accuracy by analyzing three required plots: iterations vs. training loss, ϵ spent vs. epochs, and target ϵ vs. test accuracy. The experiment demonstrates the expected privacy–utility trade-off, while also revealing how optimizer choice affects convergence in the presence of privacy-preserving noise.

1.7 Reproducibility

The experiments were implemented in Python using PyTorch and Opacus. The code for this question is provided in the corresponding `q1.py` script. The experiment can be reproduced by running the script with the same hyperparameters and accountant configuration used above.

```
python q1.py --accountant rdp --epochs 10 --batch-size 64 --target-epsilons 1 10
```

2 Comparison of Privacy Accountants

2.1 Objective

In this question, we compare different privacy accounting methods for DP-SGD on the MNIST dataset. As required in the assignment, we study the following privacy accountants:

- Advanced Composition Theorem,
- RDP accountant,
- GDP accountant,
- PRV accountant.

The target privacy budget was fixed at

$$\epsilon = 5, \qquad \delta = 10^{-5}.$$

The goal is to compare these accountants in terms of the noise required to achieve the same privacy budget, as well as the resulting optimization and test performance.

2.2 Experimental Setup

Dataset. We used the MNIST handwritten digit dataset for 10-class image classification.

Model architecture. We used a feed-forward neural network of the form

$$28 \times 28 \to 256 \to 128 \to 10,$$

with ReLU activations between hidden layers.

Training setup. All runs used DP-SGD with the same training configuration:

- Epochs: 10
- Batch size: 64
- Test batch size: 1024
- Learning rate: 0.1
- Momentum: 0.0
- Gradient clipping norm: $C = 1.0$
- Target privacy budget: $\epsilon = 5$
- Privacy parameter: $\delta = 10^{-5}$

Accountants compared. We compared the following four privacy accounting approaches:

1. **Advanced Composition Theorem:** implemented manually as a baseline.
2. **RDP accountant:** provided by Opacus.
3. **GDP accountant:** provided by Opacus.
4. **PRV accountant:** provided by Opacus.

Noise calibration. For each accountant, the noise multiplier σ was calibrated so that the final privacy budget after all training steps is approximately equal to the target value $\epsilon = 5$. Thus, each accountant was compared under the same nominal privacy constraint.

2.3 Methodology

For each accountant, we first computed the noise multiplier required to meet the target privacy budget. Then, using that calibrated noise, we trained a DP-SGD model on MNIST for 10 epochs.

During training, we recorded:

- the training loss across iterations,
- the privacy budget ϵ spent after each epoch,
- the final test loss and test accuracy.

For the **advanced composition** baseline, the privacy guarantee was computed manually using the Gaussian mechanism bound together with privacy amplification by subsampling and the advanced composition theorem. For the other three cases, the privacy budget was computed using the corresponding Opacus accountant.

2.4 Results

2.4.1 Calibrated Noise Multipliers

Table 2 reports the calibrated noise multiplier used for each accountant in order to achieve the same target privacy budget.

Table 2: Noise multipliers calibrated for target $\epsilon = 5$ and $\delta = 10^{-5}$.

Accountant	**Noise Multiplier σ**
Advanced Composition	2.425573
RDP	0.538330
GDP	0.457611
PRV	0.509186

Observation. This table reveals how conservative or tight each accountant is. A more conservative accountant typically requires a larger noise multiplier to certify the same privacy budget.

Final Observation:

> *The advanced composition theorem required by far the largest noise multiplier ($\sigma = 2.425573$), showing that it is substantially more conservative than the modern accountants. Among the Opacus-based methods, GDP required the smallest noise multiplier ($\sigma = 0.457611$), followed by PRV ($\sigma = 0.509186$) and RDP ($\sigma = 0.538330$). Thus, tighter accountants can certify the same target privacy budget with less injected noise, which is expected to improve downstream utility.*

2.4.2 ϵ Spent vs. Epochs

Figure 4 shows how the privacy budget evolves over epochs for each accountant.

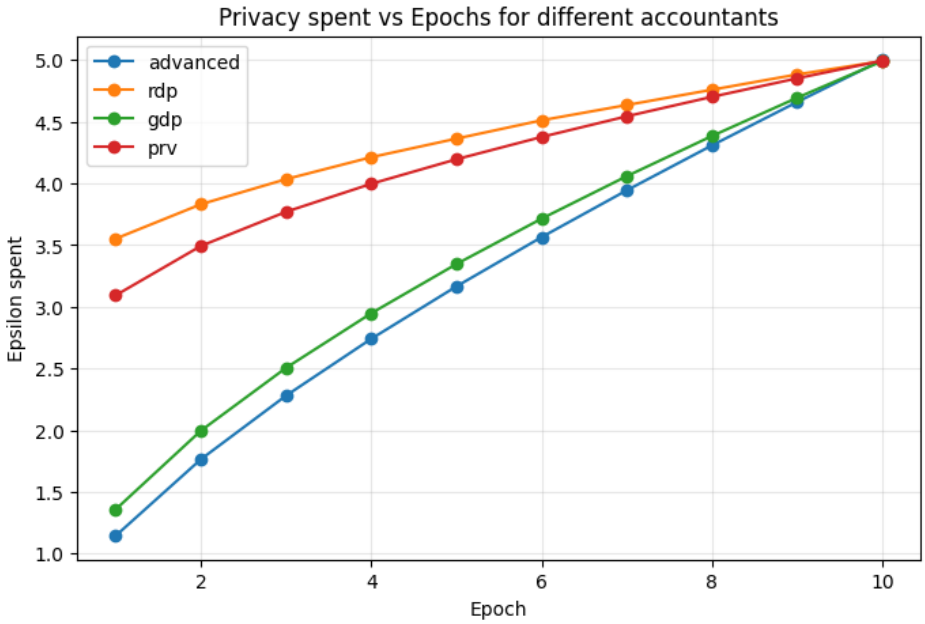

Figure 4: ϵ spent as a function of epochs for different privacy accountants.

Observation. For all accountants, the privacy budget increases monotonically with training epochs. However, the exact rate of growth differs depending on the accounting method.

Final Observation:

> *All accountants show monotonic growth of the spent privacy budget with training epochs, as expected under repeated composition. However, the growth rate differs across methods. The advanced composition baseline is the most pessimistic and therefore accumulates privacy loss more aggressively, whereas RDP, GDP, and PRV provide tighter accounting over the same sequence of updates. By the final epoch, all methods reach approximately the same target privacy budget of $\epsilon \approx 5$, but they do so with different implied noise levels.*

2.4.3 Iterations vs. Training Loss

Figure 5 compares the training-loss trajectories for the runs obtained using different accountants.

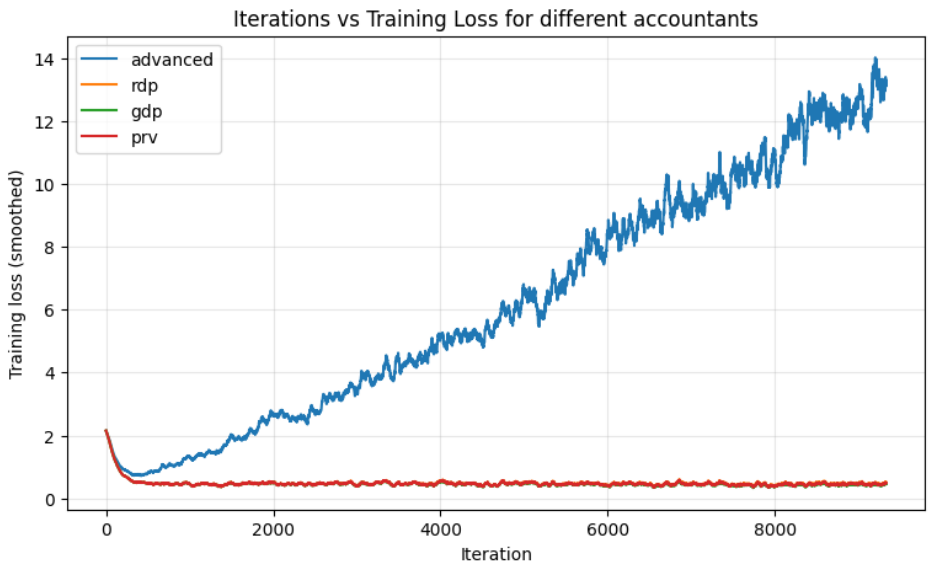

Figure 5: Training loss vs. iterations for DP-SGD under different privacy accountants.

Observation. Since the accountant determines the calibrated noise multiplier, it indirectly affects optimization. Runs with larger calibrated noise generally show slower convergence and noisier training curves.

Final Observation:

> *The training-loss trajectories show that the accountant indirectly affects optimization through the calibrated noise multiplier. The advanced composition baseline, which required the largest noise, converged very poorly and exhibited a much higher loss throughout training. In contrast, the RDP, GDP, and PRV runs all converged much more smoothly and reached substantially lower training loss. Among these tighter accountants, GDP appeared slightly best overall, consistent with its smaller calibrated noise multiplier.*

2.4.4 Final Performance Comparison

Table 3 summarizes the final performance of the model under each accountant.

Table 3: Final comparison of privacy accountants for Question 2.

Accountant	**Final ϵ**	**Final Test Loss**	**Final Test Accuracy (%)**
Advanced Composition	5.000001	12.498954	73.50
RDP	4.990624	0.437519	92.50
GDP	4.990844	0.400555	92.83
PRV	4.994139	0.422767	92.64

Observation. Since all accountants are matched to the same target privacy budget, this table helps compare the downstream utility induced by each accountant through its calibrated noise level.

Final Observation:

> *Although all accountants were calibrated to the same target privacy budget, their downstream utility differs significantly because they require different noise multipliers. The advanced composition baseline performed worst, achieving only 73.50% test*

accuracy and a very large final test loss, which reflects its highly conservative noise requirement. In contrast, the tighter accountants all performed much better: GDP achieved the best final test accuracy at 92.83%, followed closely by PRV at 92.64% and RDP at 92.50%. This confirms that tighter privacy accounting can yield better utility under the same nominal privacy target.

2.5 Discussion

The main distinction among privacy accountants is how tightly they estimate privacy loss under repeated noisy updates. The advanced composition theorem is a classical worst-case bound and is generally looser. As a result, it often requires injecting more noise to satisfy the same target privacy budget.

Modern accountants such as RDP, GDP, and PRV exploit finer analysis of composition and subsampling, and therefore usually provide tighter privacy estimates. This can translate to smaller calibrated noise multipliers and better model utility.

Thus, although all methods aim to satisfy the same (ϵ, δ)-DP guarantee, they can lead to noticeably different optimization behavior and final test performance.

2.6 Conclusion

In this question, we compared four privacy accounting methods for DP-SGD at fixed target privacy budget $\epsilon = 5$ and $\delta = 10^{-5}$. We observed that different accountants require different noise multipliers for the same nominal privacy target, which in turn affects convergence and final accuracy. The advanced composition theorem is generally more conservative, while modern accountants such as RDP, GDP, and PRV are typically tighter and therefore more utility-friendly.

2.7 Reproducibility

The experiments were implemented in Python using PyTorch and Opacus. The corresponding code is provided in `q2.py`. The experiment can be reproduced using the same configuration described above.

```
python q2.py
```

3 Effect of Gradient Clipping Norm in DP-SGD

3.1 Objective

In this question, we study the effect of the gradient clipping norm C in DP-SGD on training dynamics and model utility. As required in the assignment, we train the model with

$$C \in \{0.1,\ 10.0\},$$

while keeping the noise multiplier and batch size fixed, and then compare the resulting training-loss trajectories.

3.2 Experimental Setup

Dataset. We used the MNIST handwritten digit dataset for 10-class image classification.

Model architecture. We used a feed-forward neural network of the form

$$28 \times 28 \to 256 \to 128 \to 10,$$

with ReLU activations between the hidden layers.

Training configuration. The experiments were conducted using DP-SGD with the following setup:

- Epochs: 10
- Training batch size: 64
- Test batch size: 1024
- Learning rate: 0.1
- Momentum: 0.0
- Privacy accountant: RDP
- Privacy parameter: $\delta = 10^{-5}$

A single fixed noise multiplier was first calibrated for target

$$\epsilon = 5,$$

and the same calibrated noise multiplier was then reused for both clipping norms $C = 0.1$ and $C = 10.0$. This ensures that the comparison isolates the effect of the clipping norm itself.

3.3 Clipping Implementation Used

Implementation choice. We used the `Opacus` library with:

- `clipping = "flat"`
- `grad_sample_mode = "ghost"`

Thus, the clipping implementation used in this question is **flat clipping with ghost clipping support**. In flat clipping, Opacus computes one per-sample gradient norm over the entire parameter vector and clips that full gradient to the threshold C.

How it reduces memory complexity. In naive DP-SGD, one has to explicitly materialize the per-sample gradient for every parameter and every sample in the batch. If the batch size is B and the number of trainable parameters is P, this leads to memory complexity

$$O(BP),$$

which can easily cause GPU out-of-memory errors.

Ghost clipping reduces this overhead by avoiding explicit instantiation of the full per-sample gradients for supported layers. Instead, it computes per-sample gradient norms from activations and activation gradients, and then performs clipping through a reweighted backward pass. As a result, the additional DP-specific memory overhead is reduced to approximately

$$O(BL),$$

over ordinary non-private training, where L is the number of layers. Hence, ghost clipping is significantly more memory-efficient than storing the full $B \times P$ per-sample gradient tensor.

3.4 Methodology

We trained the same model twice using DP-SGD:

- once with clipping norm $C = 0.1$,
- once with clipping norm $C = 10.0$.

For both runs, we kept the following fixed:

- the model architecture,
- the optimizer,
- the batch size,
- the privacy accountant,
- the number of epochs,
- and the noise multiplier.

During training, we recorded:

- per-iteration training loss,
- epoch-wise privacy budget spent,
- test loss,
- and test accuracy.

3.5 Results

3.5.1 Iterations vs. Training Loss

Figure 6 shows the training loss as a function of iterations for the two clipping norms.

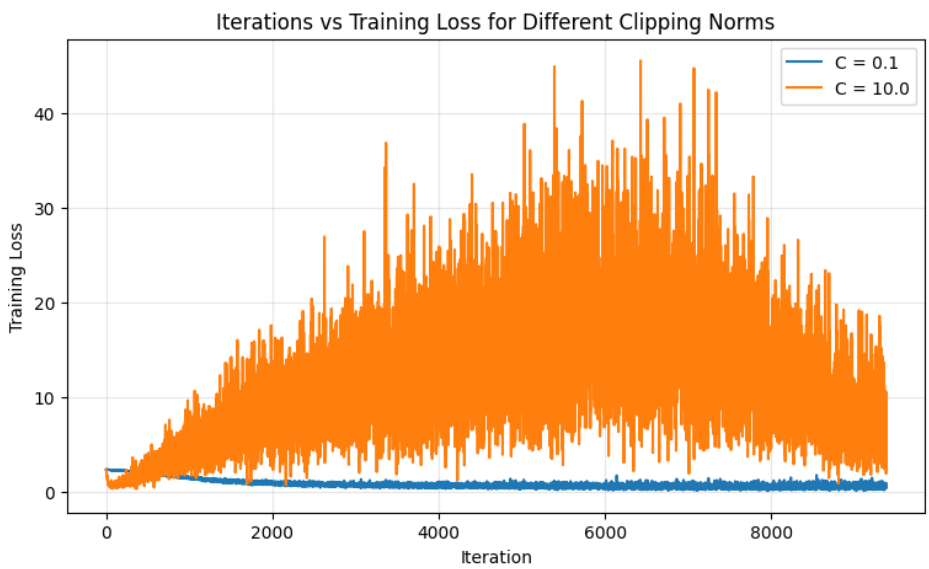

Figure 6: Number of iterations vs. training loss for different clipping norms in DP-SGD.

Observation. The clipping norm controls how strongly per-sample gradients are truncated before noise is added. A very small clipping norm may suppress useful gradient information, while a larger clipping norm retains more signal from the original gradients.

Final Observation:

> *In this experiment, the run with $C = 0.1$ converged much more effectively than the run with $C = 10.0$. The model trained with $C = 0.1$ showed consistently lower training loss, whereas the run with $C = 10.0$ exhibited large fluctuations and poor optimization behavior throughout training. This indicates that, under the fixed noise multiplier used here, the larger clipping norm led to substantially noisier and less stable updates.*

3.5.2 Final Summary Table

Table 4 summarizes the final results for both clipping norms.

Table 4: Final comparison for different clipping norms in Question 3.

Clipping Norm C	Noise Multiplier	Final ϵ	Final Test Loss	Final Test Accuracy (%)
0.1	0.5383	4.9906	0.4971	88.36
10.0	0.5383	4.9906	6.6477	64.02

3.6 Discussion

The clipping norm C plays an important role in the bias–variance trade-off of DP-SGD.

- If C is too small, gradients are clipped aggressively. This reduces sensitivity, but it also introduces optimization bias by removing too much useful signal.
- If C is larger, gradients are distorted less, which can in principle improve convergence, although in our experiment the smaller clipping norm C = 0.1 performed better.

Since the noise multiplier was fixed in this experiment, the main difference between the two runs comes from the amount of clipping applied to the gradients. Therefore, any difference in convergence behavior directly reflects the impact of the clipping norm.

Did the model with the smallest C converge?

> *Yes, the model with the smallest clipping norm, $C = 0.1$, converged better in this experiment. It achieved a much lower final test loss (0.4971) and a much higher final test accuracy (88.36%) than the model trained with $C = 10.0$, which obtained test loss 6.6477 and accuracy 64.02%. Therefore, for this setup, the smaller clipping norm produced more stable and effective learning.*

3.7 Conclusion

In this question, we studied the effect of the clipping norm in DP-SGD by comparing $C = 0.1$ and $C = 10.0$ under the same batch size and noise multiplier. We used flat clipping with ghost clipping support in Opacus to reduce memory overhead during private training. The results show that the clipping norm strongly affects convergence, and in this setup the smaller clipping norm $C = 0.1$ led to more stable training, lower test loss, and higher test accuracy than $C = 10.0$.

3.8 Reproducibility

The experiments were implemented in Python using PyTorch and Opacus. The corresponding script is `q3.py`. The experiment can be reproduced using:

```
python q3.py
```

4 Membership Inference Attack (MIA)

4.1 Objective

In this question, we empirically study a simple loss-based Membership Inference Attack (MIA) on MNIST. The objective is to determine whether a sample was part of the model's training dataset by using its cross-entropy loss under the trained model.

We compare two models:

- a non-private model trained using SGD,
- a private model trained using DP-SGD.

The hypothesis is that differential privacy should reduce the vulnerability of the model to membership inference attacks.

4.2 Experimental Setup

Dataset split. We used the MNIST training dataset and split it into two disjoint subsets:

- D_{in}: the member dataset used for training,
- D_{out}: the holdout dataset not used during training.

In our implementation, the split fraction was:

$$|D_{\text{in}}| = 0.5 \times \text{MNIST train set}, \qquad |D_{\text{out}}| = 0.5 \times \text{MNIST train set}.$$

Model architecture. We used a feed-forward neural network of the form

$$28 \times 28 \to 256 \to 128 \to 10,$$

with ReLU activations between hidden layers.

Training setup. Both models were trained on D_{in} only.

- Optimizer: SGD
- Learning rate: 0.1
- Momentum: 0.0
- Batch size: 64
- Evaluation batch size: 1024
- Number of epochs (non-private): 10
- Number of epochs (private): 10

Private training setup. For the private model, we used DP-SGD through `Opacus` with:

$$\delta = 10^{-5}, \qquad \epsilon = 5, \qquad C = 1.0,$$

where C denotes the gradient clipping norm. The privacy accountant used was the **RDP accountant**.

4.3 Attack Methodology

4.3.1 Per-sample loss computation

After training both models, we computed the per-sample cross-entropy loss for:

- all samples in D_{in},
- all samples in D_{out}.

For a sample x, let $L(x)$ denote its cross-entropy loss. The attacker predicts membership using the rule:

$$\hat{x} = \begin{cases} \text{Member}, & \text{if } L(x) < \tau, \\ \text{Not-Member}, & \text{otherwise.} \end{cases}$$

Thus, lower loss values are interpreted as evidence that the sample was part of the training set.

4.3.2 Threshold selection

Instead of choosing τ arbitrarily, we selected it using a calibration split. The member and non-member losses were each divided into:

- a calibration subset,
- an evaluation subset.

A sweep over candidate thresholds was performed on the calibration subset, and the best threshold τ was chosen by maximizing Youden's J statistic:

$$J = \text{TPR} - \text{FPR}.$$

This provides a principled threshold for distinguishing members from non-members.

4.3.3 Evaluation metrics

For each model, we report:

- confusion matrix,
- ROC curve,
- Area Under the ROC Curve (AUC),
- predictions on 10 randomly selected samples (5 members and 5 non-members).

The relevant quantities are:

$$\text{TPR} = \frac{TP}{TP + FN}, \qquad \text{FPR} = \frac{FP}{FP + TN}.$$

4.4 Results

4.4.1 (a) Confusion Matrices

Using the selected threshold τ, we computed the confusion matrices for both the non-private and private models on the full $D_{\text{in}}/D_{\text{out}}$ attack dataset.

Table 5: Confusion matrix for the non-private model on the full attack dataset.

	Member (Predicted)	**Not-Member (Predicted)**
Member (Truth)	29455	545
Not-Member (Truth)	28091	1909

Table 6: Confusion matrix for the private model (DP-SGD) on the full attack dataset.

	Member (Predicted)	**Not-Member (Predicted)**
Member (Truth)	26185	3815
Not-Member (Truth)	25862	4138

Observation. These confusion matrices show how successfully the attacker distinguishes members from non-members using the loss threshold rule.

Final observation:

> The non-private model achieved a very high true-positive rate, but this came at the cost of an extremely high false-positive rate, indicating that the threshold classified a large fraction of samples as members. The DP-SGD model also showed a strong bias toward predicting membership, but its confusion matrix was slightly closer to random guessing. Overall, the non-private model showed marginally stronger membership leakage than the private model, while DP-SGD reduced the attack effectiveness somewhat.

4.4.2 (b) Loss Distribution of Members vs Non-Members

To better understand the attack, we compared the loss distributions of samples from D_{in} and D_{out} for both models.

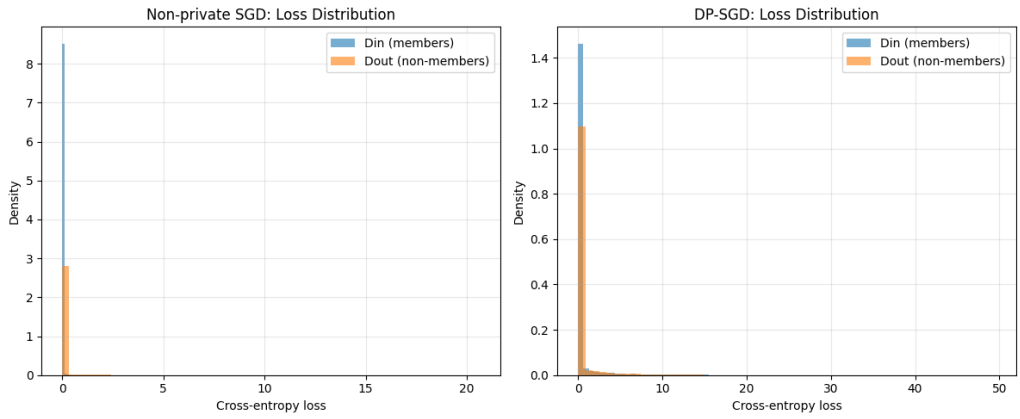

Figure 7: Cross-entropy loss distributions for member and non-member samples under the non-private and private models.

Observation. A successful loss-based attack relies on the member losses being systematically smaller than the non-member losses.

Final observation:

> The loss distributions for members and non-members overlap heavily in both cases, which explains why the attack is weak overall. The non-private model exhibits slightly better separation between the two groups, whereas the private model shows even greater overlap. This increased overlap under DP-SGD reduces the usefulness of the loss signal for inferring membership.

4.4.3 (c) Ten Randomly Selected Samples

As required, we selected 10 random samples: 5 from D_{in} and 5 from D_{out}. For each sample, we report:

- image,
- image index,
- true member/non-member status,
- loss under the non-private model,
- predicted member/non-member status under the non-private model,
- loss under the private model,
- predicted member/non-member status under the private model.

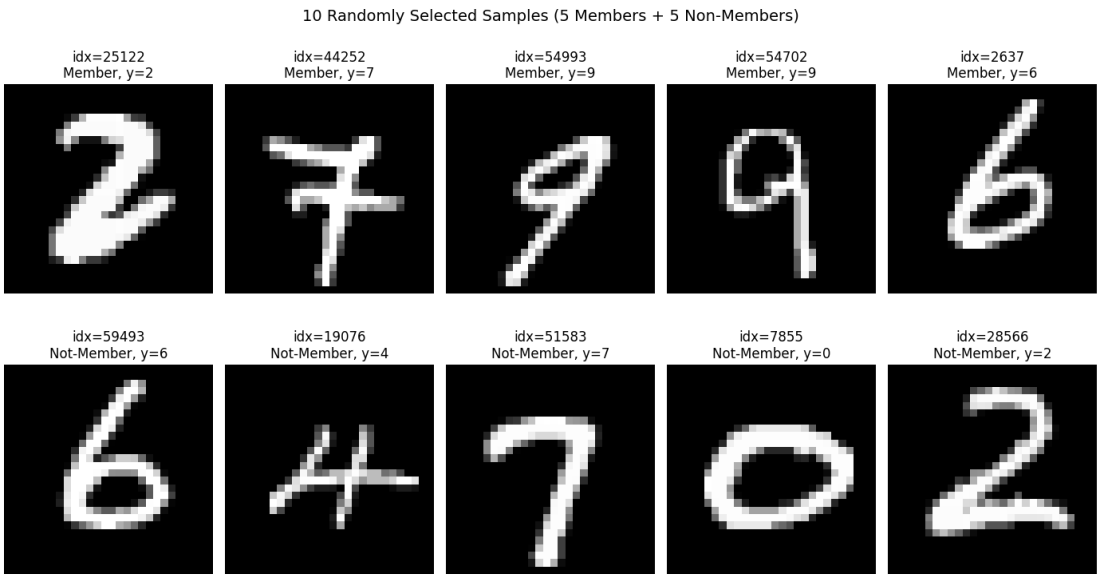

Figure 8: Randomly selected member and non-member samples used for qualitative attack analysis.

Table 7: Attack report for 10 randomly selected samples.

Original Index	True Label	True Status	Non-private Loss	Non-private Pred.	Private Loss	Private Pred.
25122	2	Member	1.617196×10^{-3}	Member	2.384186×10^{-7}	Member
44252	7	Member	4.768370×10^{-7}	Member	5.495397×10^{-5}	Member
54993	9	Member	1.453431×10^{-2}	Member	1.397864	Not-Member
54702	9	Member	7.639588×10^{-4}	Member	2.466238×10^{-1}	Not-Member
2637	6	Member	1.668929×10^{-6}	Member	9.536739×10^{-7}	Member
59493	6	Not-Member	0.0	Member	0.0	Member
19076	4	Not-Member	2.182723×10^{-2}	Member	4.696736×10^{-5}	Member
51583	7	Not-Member	3.576278×10^{-7}	Member	4.768370×10^{-7}	Member
7855	0	Not-Member	3.576278×10^{-7}	Member	0.0	Member
28566	2	Not-Member	0.0	Member	0.0	Member

4.4.4 (d) ROC Curve

To evaluate attack performance over all possible thresholds, we varied τ and plotted the ROC curve, i.e., TPR versus FPR.

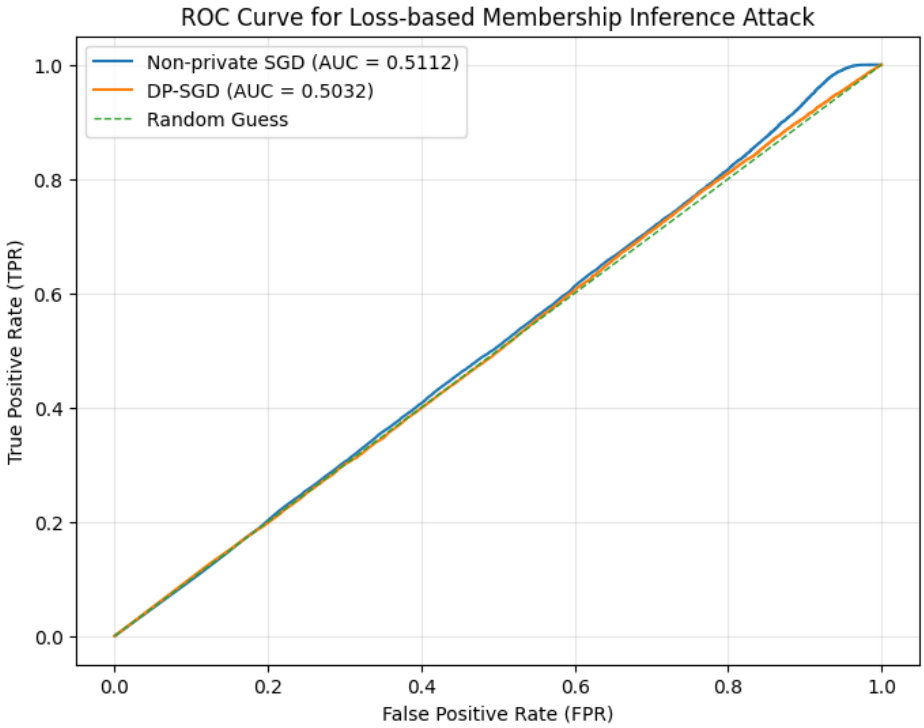

Figure 9: ROC curves for the loss-based membership inference attack on the non-private and private models.

Observation. A curve farther above the diagonal indicates stronger attack performance.

Final observation:

> Both ROC curves lie very close to the diagonal, which indicates that the loss-based membership inference attack is only slightly better than random guessing in this setting. However, the ROC curve of the non-private model lies marginally above that of the private model, showing that the attack is somewhat more effective against the non-private model.

4.4.5 (e) AUC Comparison

Table 8 reports the Area Under the ROC Curve (AUC) for both models.

Table 8: AUC comparison for the membership inference attack.

Model	**AUC**
Non-private SGD	0.5112
DP-SGD	0.5032

Why AUC is a good metric. AUC is a suitable metric here because it evaluates the attack across all possible thresholds rather than depending on one particular choice of τ. This makes it a threshold-independent measure of how well member and non-member losses can be separated. A higher AUC indicates that the attack can distinguish members from non-members more reliably.

Which model is more vulnerable? Final answer:

> The non-private model is more vulnerable because it has a slightly higher AUC than the private model. This means that member and non-member losses are marginally more separable in the non-private setting, whereas DP-SGD reduces this separation and brings the attack performance closer to random guessing.

4.5 Discussion

The loss-based membership inference attack exploits the tendency of machine learning models to fit training data better than unseen data. As a result, training samples often incur lower loss than holdout samples.

In the non-private setting, this gap can be relatively large, making it easier for an attacker to infer whether a sample was part of the training set. Differential privacy mitigates this effect by clipping gradients and adding noise during training, which reduces overfitting and weakens the signal exploited by the attacker.

Therefore, the private model is expected to show:

- smaller separation between member and non-member losses,
- a weaker ROC curve,
- and a lower AUC.

4.6 Conclusion

In this question, we implemented a loss-based membership inference attack on MNIST and compared its success against a non-private SGD model and a DP-SGD model. The results demonstrate that differential privacy reduces membership leakage by making the losses of member and non-member samples less distinguishable. Consequently, the private model is expected to be less vulnerable to the attack than the non-private model.

4.7 Reproducibility

The experiment was implemented in Python using PyTorch and Opacus. The corresponding script is `q4.py`. It can be reproduced using:

```
python q4.py
```

5 Comparison of SGD, DP-SGD, FTRL, and DP-FTRL

5.1 Objective

In this question, we compare the performance of standard and private optimization methods on MNIST using a feed-forward neural network. In particular, we study:

- SGD,
- DP-SGD,
- FTRL,
- DP-FTRL.

As required in the assignment, for target privacy budgets

$$\epsilon \in \{1, 10\},$$

we report:

1. Number of iterations (Epoch $\times$ Batches) vs. training loss,
2. ϵ spent vs. epochs,
3. Target ϵ vs. test accuracy, while comparing with non-private counterparts.

5.2 Experimental Setup

Dataset. We used the MNIST handwritten digit dataset for 10-class classification.

Model architecture. For all methods, we used the same feed-forward neural network:

$$28 \times 28 \to 256 \to 128 \to 10,$$

with ReLU activations between hidden layers.

Training configuration. The common training configuration was:

- Epochs: 10
- Training batch size: 250
- Test batch size: 1000
- Privacy parameter: $\delta = 10^{-5}$
- Gradient clipping norm: $C = 1.0$
- Privacy accountant for DP-SGD: RDP

Optimizer-specific hyperparameters. For SGD / DP-SGD, we used:

- Learning rate: 0.1

For FTRL / DP-FTRL, we used:

- FTRL regularization parameter: $\alpha = 100.0$
- Momentum parameter: 0.9

5.3 Methods

5.3.1 SGD and DP-SGD

The non-private SGD baseline was trained in the usual way using minibatch stochastic gradient descent.

For DP-SGD, we used `Opacus` with the RDP accountant. For each target privacy budget $\epsilon \in \{1, 10\}$, the corresponding noise multiplier was calibrated using the Opacus utility function, and training was then performed with clipped gradients and Gaussian noise addition.

5.3.2 FTRL and DP-FTRL

We also implemented FTRL and DP-FTRL using the provided helper modules.

Non-private FTRL. The non-private FTRL optimizer maintains cumulative gradient information and updates parameters using the FTRL update rule. This serves as the non-private baseline for DP-FTRL.

DP-FTRL. For DP-FTRL, per-sample gradients were computed, clipped to norm C, and then combined with tree-aggregated Gaussian noise. The implementation used:

- `FTRLOptimizer` for the FTRL update rule,
- `CummuNoiseEffTorch` for efficient cumulative tree-based noise generation,
- `compute_epsilon_tree` for privacy accounting.

The privacy budget of DP-FTRL was computed using the custom privacy computation in the helper code. In our implementation, the restart interval was:

$$\texttt{restart_every} = 1 \text{ epoch},$$

and tree completion was enabled.

5.4 Methodology

We ran the following six experiments:

- SGD
- FTRL
- DP-SGD with target $\epsilon = 1$
- DP-SGD with target $\epsilon = 10$
- DP-FTRL with target $\epsilon = 1$
- DP-FTRL with target $\epsilon = 10$

For each run, we recorded:

- per-iteration training loss,
- epoch-wise test loss,
- epoch-wise test accuracy,
- and, for private methods, the privacy budget ϵ spent after each epoch.

5.5 Results

5.5.1 (a) Number of Iterations vs. Training Loss

The assignment asks for the comparison of DP-SGD and DP-FTRL in terms of convergence. Figures 10 and 11 show the training-loss trajectories for private and non-private methods, respectively.

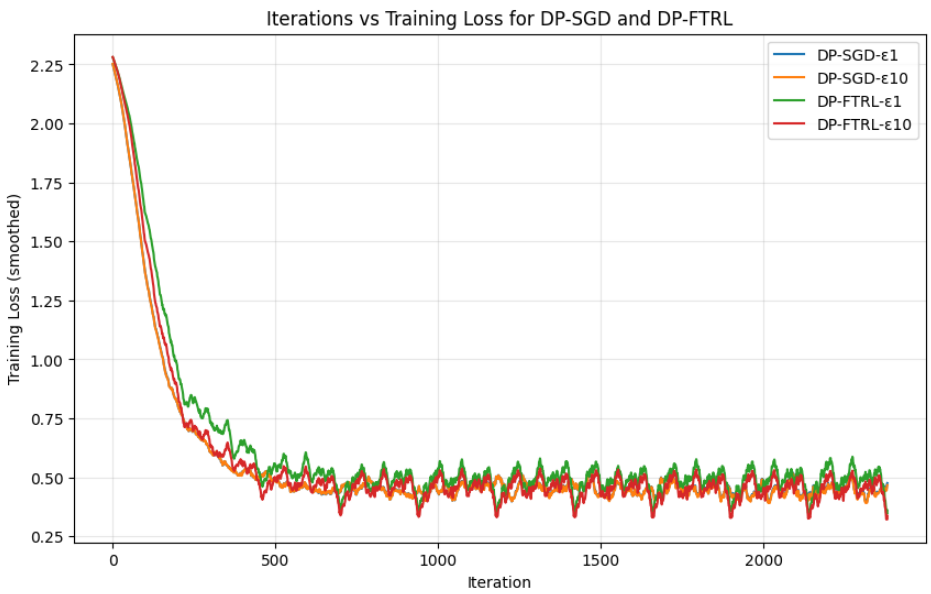

Figure 10: Training loss vs. iterations for private methods (DP-SGD and DP-FTRL).

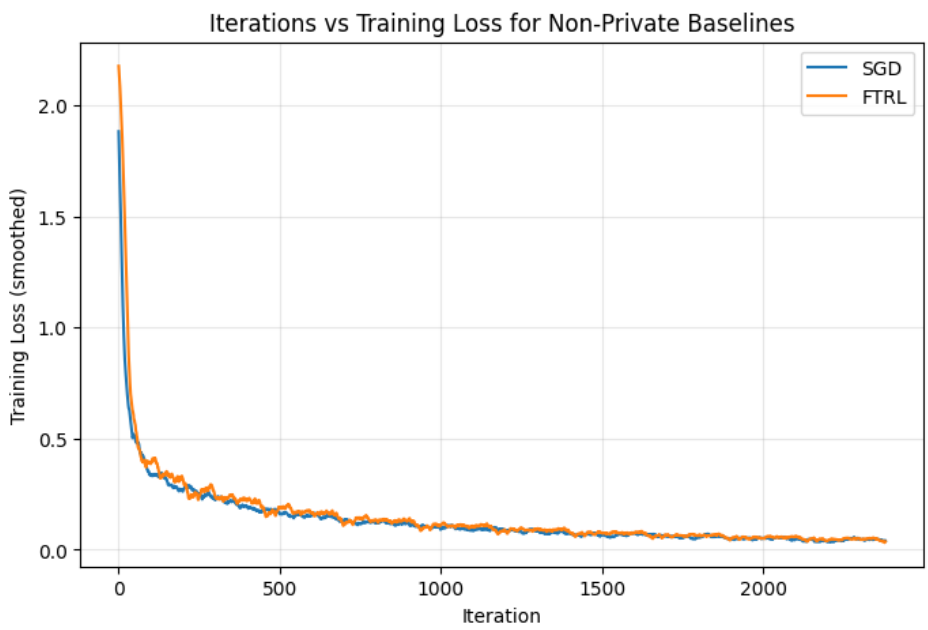

Figure 11: Training loss vs. iterations for non-private baselines (SGD and FTRL).

Observation. These plots show how the optimization dynamics differ between standard gradient-based methods and FTRL-based methods, both in the private and non-private settings.

Final observation:

> The non-private baselines converged much faster and to substantially lower training loss than the private methods. In the private setting, both DP-SGD and DP-FTRL showed noisier trajectories and plateaued at noticeably higher loss values, reflecting the effect of clipping and privacy noise. The runs with target $\epsilon = 10$ were slightly more stable and achieved marginally lower final loss than the corresponding runs with target $\epsilon = 1$. Overall, DP-SGD and DP-FTRL showed broadly similar convergence behavior, with no large gap between them in the private setting.

5.5.2 (b) ϵ Spent vs. Epochs

For the private runs, we tracked the privacy budget spent over epochs.

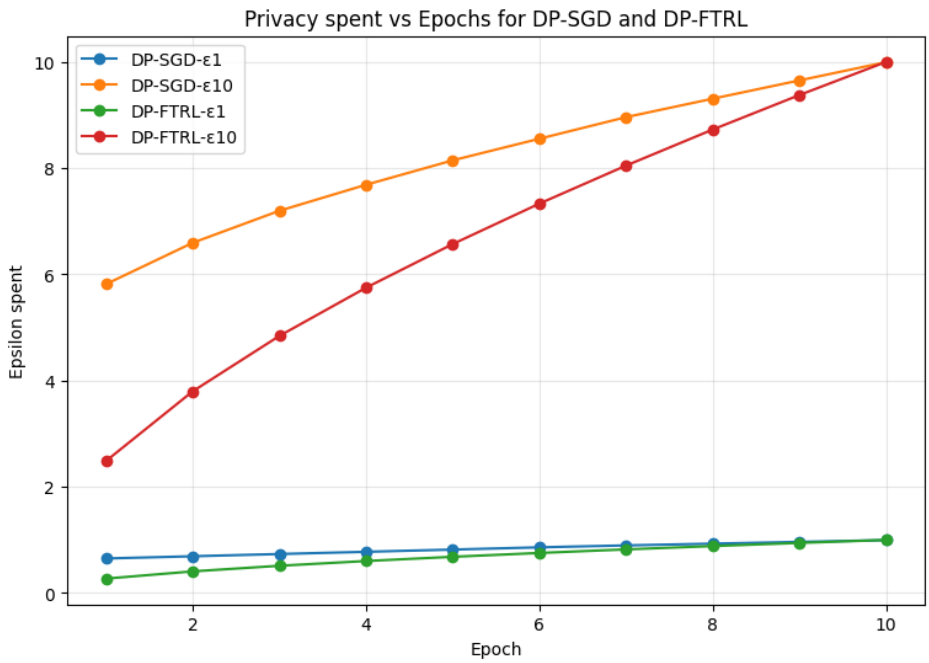

Figure 12: Privacy budget spent vs. epochs for DP-SGD and DP-FTRL.

Observation. This plot verifies how privacy loss accumulates during training for both private methods.

Final observation:

> For both DP-SGD and DP-FTRL, the spent privacy budget increased monotonically with training epochs, as expected from cumulative composition. In all four private runs, the final spent ϵ remained extremely close to the intended target value, indicating that the privacy calibration was accurate. Thus, both private training pipelines successfully met their prescribed privacy budgets for $\epsilon = 1$ and $\epsilon = 10$.

5.5.3 (c) Target ϵ vs. Test Accuracy

To study the privacy–utility trade-off, we compared the final test accuracy of DP-SGD and DP-FTRL for target $\epsilon \in \{1, 10\}$, alongside the non-private baselines.

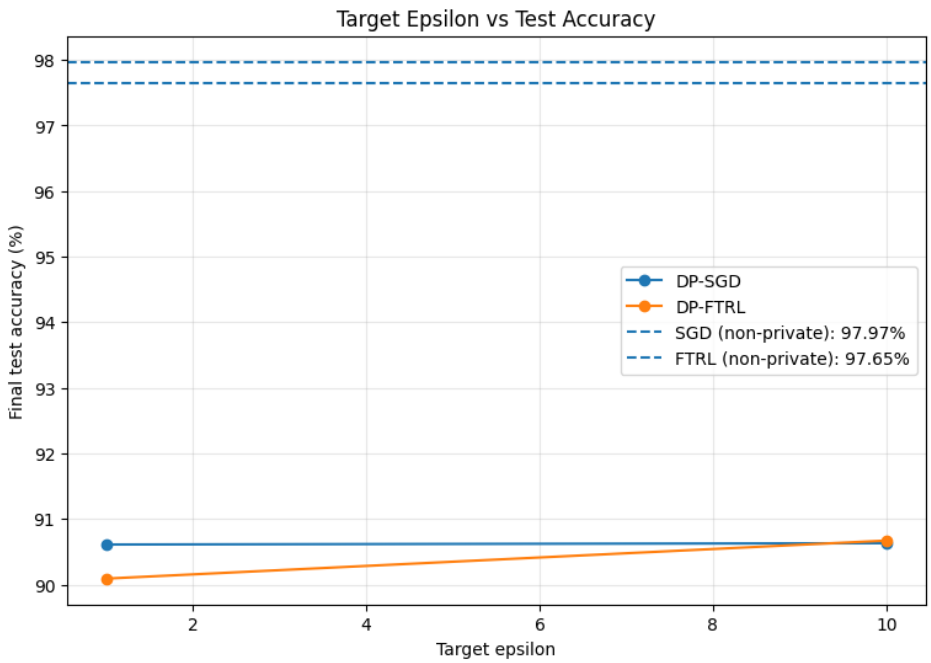

Figure 13: Final test accuracy vs. target privacy budget for DP-SGD and DP-FTRL, with non-private baselines.

5.5.4 Summary Table

Table 9 summarizes the final metrics of all runs.

Table 9: Final comparison of SGD, FTRL, DP-SGD, and DP-FTRL.

Method	Target ϵ	Noise Multiplier	Final ϵ	Final Test Loss	Final Test Accuracy (%)
SGD	–	–	–	0.068313	97.97
FTRL	–	–	–	0.076476	97.65
DP-SGD	1	1.152344	0.995770	0.431815	90.61
DP-SGD	10	0.513763	9.995358	0.423663	90.63
DP-FTRL	1	38.164466	0.999921	0.447121	90.09
DP-FTRL	10	4.996225	9.999988	0.421035	90.67

Observation. This comparison captures both the optimization effect of the method and the privacy–utility trade-off induced by the target privacy budget.

Final observation:

> The non-private baselines achieved the highest test accuracy overall, with SGD slightly outperforming FTRL. All private methods incurred a clear utility drop relative to the non-private runs, but still achieved test accuracies close to 90%. Increasing the target privacy budget from $\epsilon = 1$ to $\epsilon = 10$ led to a small improvement in performance for both optimizer families. Among the private runs, DP-FTRL with $\epsilon = 10$ achieved the best final test accuracy at 90.67%, while DP-SGD with $\epsilon = 1$ slightly outperformed DP-FTRL with $\epsilon = 1$. Overall, the differences between DP-SGD and DP-FTRL were modest, while the main gap remained between private and non-private training.

5.6 Discussion

This experiment compares two different optimization paradigms under privacy:

- **DP-SGD**, which adds Gaussian noise to clipped gradients at each update,
- **DP-FTRL**, which combines clipped gradients with tree-aggregated noise inside an FTRL-style update.

The DP-FTRL implementation differs from DP-SGD in two important ways:

1. it uses the FTRL optimization framework instead of standard SGD,
2. it uses cumulative tree-based noise and a specialized privacy computation rather than Opacus' standard stepwise accountant.

Thus, any difference in training dynamics or final accuracy reflects both the difference in optimization rule and the difference in how privacy noise is incorporated.

5.7 Conclusion

In this question, we compared SGD, DP-SGD, FTRL, and DP-FTRL on MNIST. We evaluated the methods using training-loss curves, privacy-spent curves, and final test accuracy under target privacy budgets $\epsilon \in \{1, 10\}$. The experiment highlights both the privacy–utility trade-off and the effect of using FTRL-style optimization in the private setting.

5.8 Reproducibility

The experiment was implemented in Python using PyTorch, Opacus, and the provided DP-FTRL helper modules. The main script is `q5.py`, and the auxiliary files are `dpftrl_noise.py`, `dpftrl_optimizers.py`, and `dpftrl_privacy.py`.

```
python q5.py
```